

CSE 464 Project Part 3 README

1. Project Overview

For Part 3, I refactored my graph search code from the previous project parts and added design patterns to make the code cleaner and easier to manage.

The project now supports:

- BFS
- DFS
- Random Walk

The graph is loaded from a DOT file, and the traversal history gets printed in the console while the searches run.

The main goal of this part was to reduce duplicate code and organize the search logic better.

2. How to Compile and Run

To compile the project:

- mvn clean package

To run the project:

- java -cp target/classes edu.asu.cse464.Main

3. Input Graph Used

The demo graph used for testing is:

```
digraph {  
a -> b;  
b -> c;  
c -> d;  
d -> a;  
a -> e;  
e -> f;  
e -> g;  
f -> h;  
g -> h;  
}
```

The searches were mainly tested from node a to node h.

4. Five Refactoring's Completed

- Added a helper method called getLastNode() in the Path class
- Updated getNodes() to return a copy of the list instead of the original one
- Created GraphSearchTemplate to move shared search logic into one place
- Split BFS and DFS into separate strategy classes
- Added RandomWalkStrategy and connected it to GraphSearch

5. Template Pattern Explanation

I used the Template Pattern by creating the GraphSearchTemplate class.

The common search steps are handled in this class so BFS, DFS, and Random Walk do not repeat the same logic over and over again.

The subclasses only handle the parts that are different, like how nodes are removed from the frontier or how the next paths are selected.

6. Strategy Pattern Explanation

I used the Strategy Pattern to choose which search algorithm to run.

Inside GraphSearch, the program creates a different strategy object depending on the selected algorithm:

- BFSSStrategy
- DFSSStrategy
- RandomWalkStrategy

This made the search code cleaner because everything runs through the same GraphSearch method.

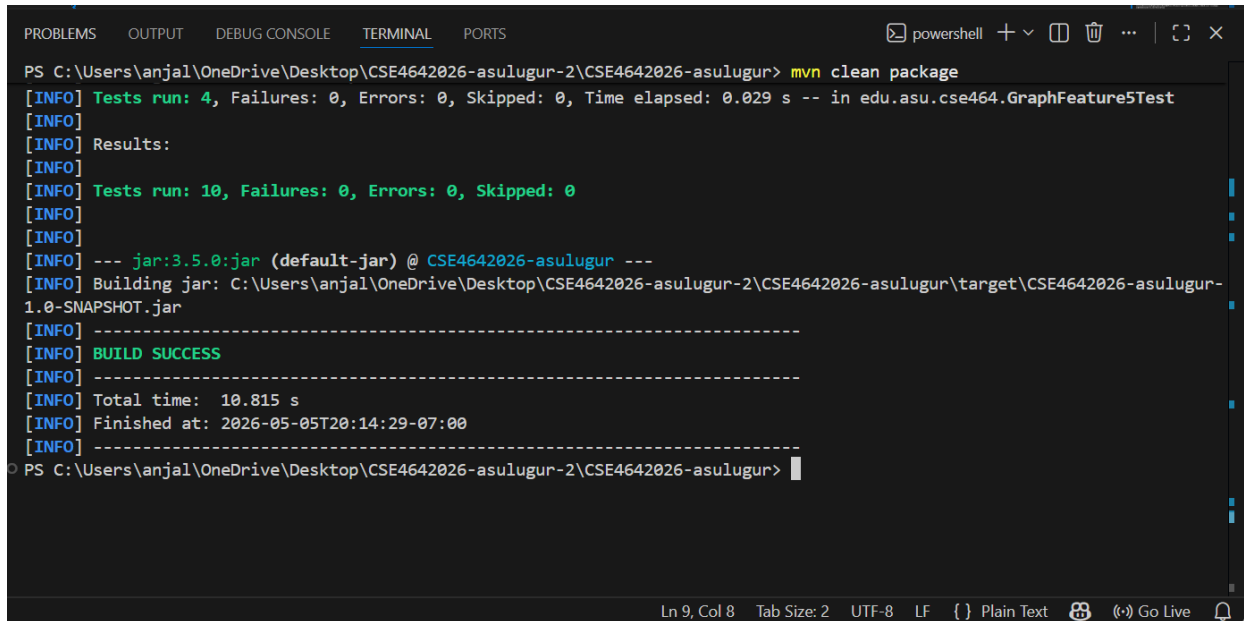
7. Random Walk Explanation

Random Walk was added as another search strategy.

Instead of checking every possible neighbor like BFS or DFS, it randomly chooses one unvisited neighbor at a time.

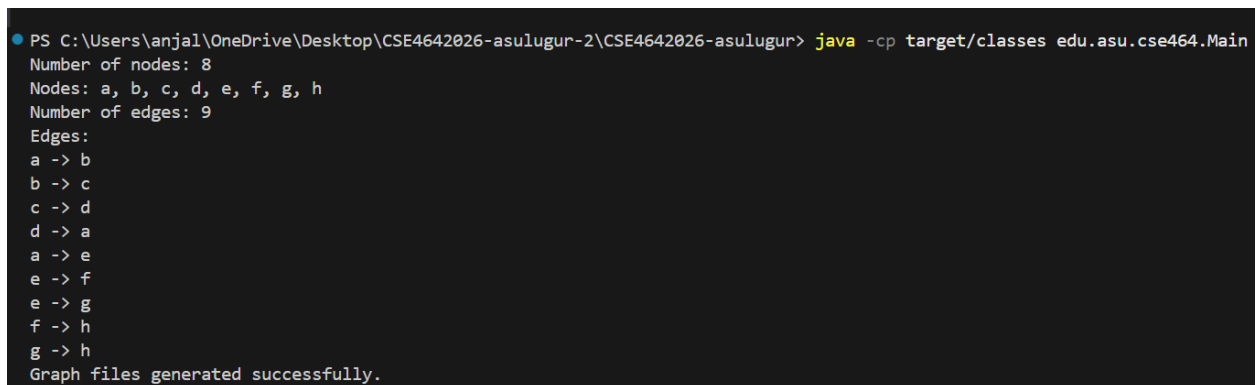
I ran it multiple times to show that the path can change because of randomness.

8. Expected Output Screenshots



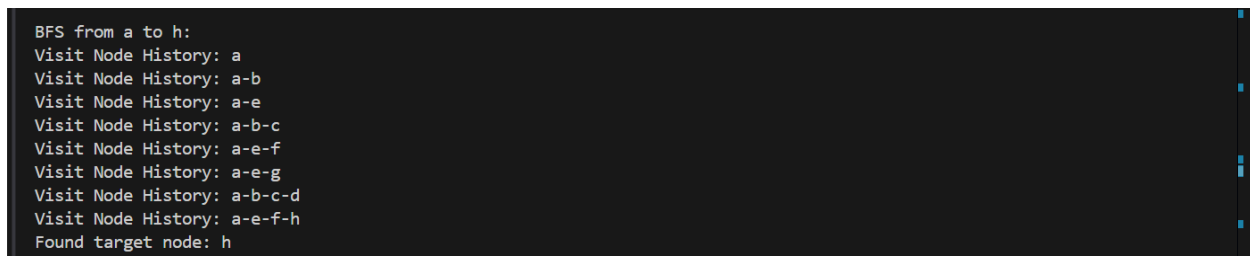
```
PS C:\Users\anjali\OneDrive\Desktop\CSE4642026-asulugur-2\CSE4642026-asulugur> mvn clean package
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.029 s -- in edu.asu.cse464.GraphFeature5Test
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 10, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] --- jar:3.5.0:jar (default-jar) @ CSE4642026-asulugur ---
[INFO] Building jar: C:\Users\anjali\OneDrive\Desktop\CSE4642026-asulugur-2\CSE4642026-asulugur\target\CSE4642026-asulugur-1.0-SNAPSHOT.jar
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 10.815 s
[INFO] Finished at: 2026-05-05T20:14:29-07:00
[INFO] -----
PS C:\Users\anjali\OneDrive\Desktop\CSE4642026-asulugur-2\CSE4642026-asulugur>
```

Figure 1: Maven build success and all tests passing



```
PS C:\Users\anjali\OneDrive\Desktop\CSE4642026-asulugur-2\CSE4642026-asulugur> java -cp target/classes edu.asu.cse464.Main
Number of nodes: 8
Nodes: a, b, c, d, e, f, g, h
Number of edges: 9
Edges:
a -> b
b -> c
c -> d
d -> a
a -> e
e -> f
e -> g
f -> h
g -> h
Graph files generated successfully.
```

Figure 2: input.dot

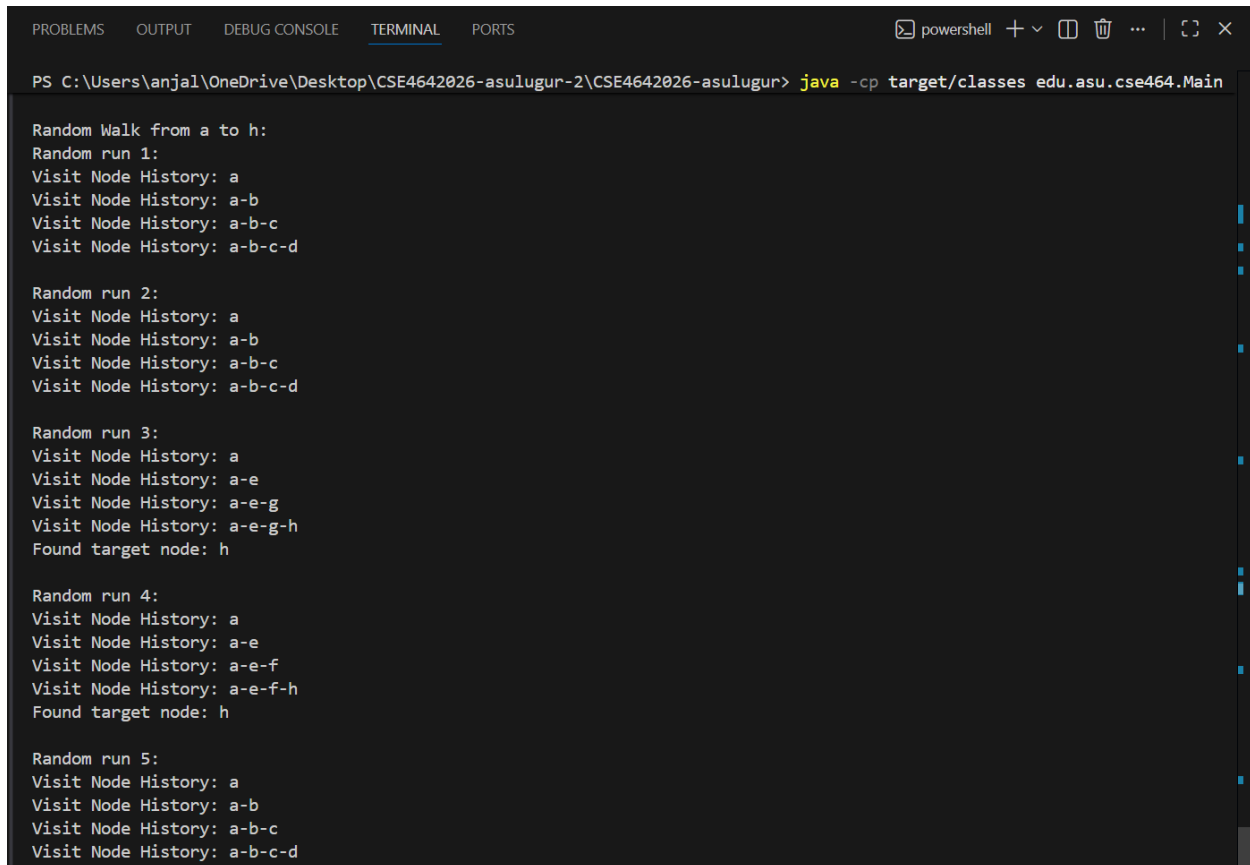


```
BFS from a to h:
Visit Node History: a
Visit Node History: a-b
Visit Node History: a-e
Visit Node History: a-b-c
Visit Node History: a-e-f
Visit Node History: a-e-g
Visit Node History: a-b-c-d
Visit Node History: a-e-f-h
Found target node: h
```

Figure 3: BFS traversal output

```
DFS from a to h:  
Visit Node History: a  
Visit Node History: a-e  
Visit Node History: a-e-g  
Visit Node History: a-e-g-h  
Found target node: h
```

Figure 4: DFS traversal output



```
PS C:\Users\anjali\OneDrive\Desktop\CSE4642026-asulugur-2\CSE4642026-asulugur> java -cp target/classes edu.asu.cse464.Main  
  
Random Walk from a to h:  
Random run 1:  
Visit Node History: a  
Visit Node History: a-b  
Visit Node History: a-b-c  
Visit Node History: a-b-c-d  
  
Random run 2:  
Visit Node History: a  
Visit Node History: a-b  
Visit Node History: a-b-c  
Visit Node History: a-b-c-d  
  
Random run 3:  
Visit Node History: a  
Visit Node History: a-e  
Visit Node History: a-e-g  
Visit Node History: a-e-g-h  
Found target node: h  
  
Random run 4:  
Visit Node History: a  
Visit Node History: a-e  
Visit Node History: a-e-f  
Visit Node History: a-e-f-h  
Found target node: h  
  
Random run 5:  
Visit Node History: a  
Visit Node History: a-b  
Visit Node History: a-b-c  
Visit Node History: a-b-c-d
```

Figure 5: Random Walk output showing different successful paths

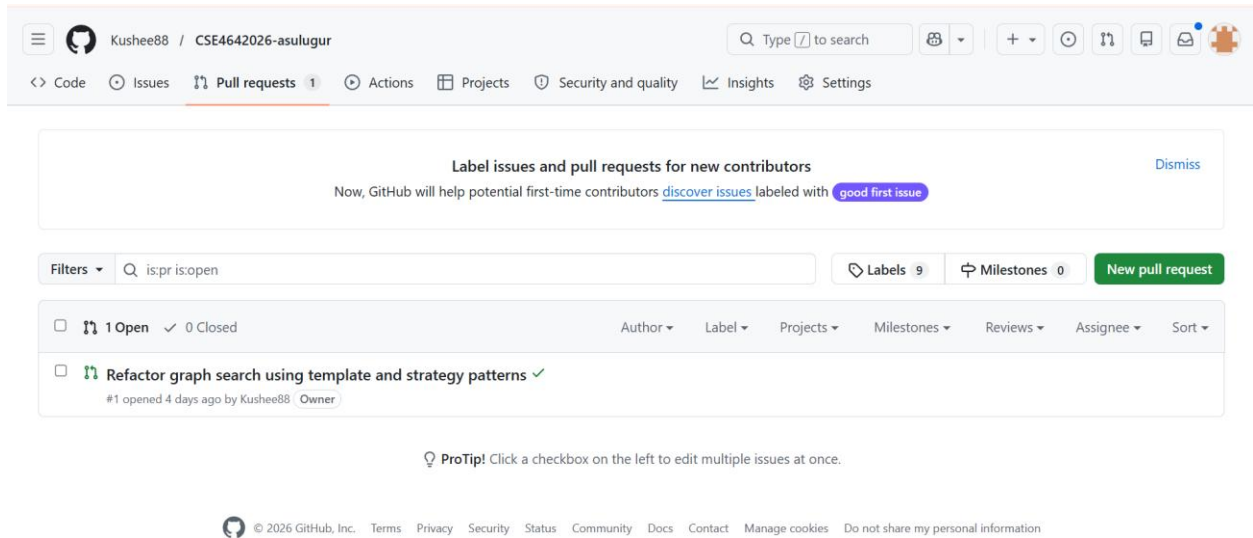


Figure 6: Pull request page showing refactor branch changes



Figure 7: GitHub commit history showing refactoring and design pattern implementation

Kushee88 started a reviewPending[View reviewed changes](#)

src/main/java/edu/asu/cse464/Graph.java

10 +

8 11 //using sets so duplicate edges do not get added

9 12 private final Set<String> nodes;

10 13 private final Set<Edge> edges;

Kushee88PendingOwnerAuthor...

adding comments or documentation explaining why Set is used here instead of List would be great, so future readers understand the design decision more clearly.





src/main/java/edu/asu/cse464/Graph.java

12 15 //used to choose which search algorithm to run

13 16 public enum Algorithm {

14 - BFS, DFS

17 + BFS, DFS, RANDOM_WALK

Kushee88PendingOwnerAuthor...

it might be helpful to add a short description for each algorithm in the enum so it is clearer what each option represents when used in GraphSearch.





src/main/java/edu/asu/cse464/GraphSearchTemplate.java

16 +

17 + // main search logic used by bfs, dfs, and random walk

18 + public Path search(String src, String dst) {

19 +

Kushee88PendingOwnerAuthor...

this method is doing a lot of things like traversal, tracking visited nodes, and building the path. It might be better to split this into smaller helper methods so it is easier to read and manage.





src/main/java/edu/asu/cse464/Main.java

37 + System.out.println("Random Walk from a to h:");

38 +

39 + //running random walk multiple times to show different random paths

40 + for (int i = 1; i <= 7; i++) {

Kushee88PendingOwnerAuthor...

right now the number of random walk runs is hardcoded. It might be better to make this configurable so it is easier to test different cases without changing the code every time.





src/main/java/edu/asu/cse464/Graph.java

249 265 //final search API that lets user choose bfs or dfs

250 266 public Path GraphSearch(String src, String dst, Algorithm algo) {

267 + GraphSearchTemplate strategy;

268 +

Kushee88PendingOwnerAuthor...

right now the algorithm selection is handled here directly. It might be better to separate this into a dedicated class or method so the code is cleaner and easier to extend later.





Figure 8: Pull request review comments and discussion

9. GitHub Commit Links

<https://github.com/Kushee88/CSE4642026-asulugur/commit/9f11b36bb69b455f0e9b7990daf56ae895a283cb>

<https://github.com/Kushee88/CSE4642026-asulugur/commit/c83a5f1d35f6a0d151026310d0be844f8bb040ee>

<https://github.com/Kushee88/CSE4642026-asulugur/commit/a2e772090f44d962de0ec8906db4b0ecaeeebfa9>

<https://github.com/Kushee88/CSE4642026-asulugur/commit/b9ed7369cfc6a2b340041d93682e39a814305497>

<https://github.com/Kushee88/CSE4642026-asulugur/commit/13f93d8a6a30ff860f7f1a00192e658e8fc238ff>

<https://github.com/Kushee88/CSE4642026-asulugur/commit/cacb400194ca91dc2cfdfa45d42489cbc333a9e5>

<https://github.com/Kushee88/CSE4642026-asulugur/commit/1a1495a073b0d32934efc80b666a840d4961e278>

<https://github.com/Kushee88/CSE4642026-asulugur/commit/8b012c5e5d78cbbf88d501db9a8a326f0fed1697>

<https://github.com/Kushee88/CSE4642026-asulugur/commit/098187781c1010a2516c455e9d3e926fa2dea856>

<https://github.com/Kushee88/CSE4642026-asulugur/commit/9174ba349226bd32a142d107db3328014e6469b9>

<https://github.com/Kushee88/CSE4642026-asulugur/commit/c5372893903d804510eb186745fc8ab45d65ea55>

10. Pull Request and Code Review

I created a pull request from the refactor branch into main after finishing the refactoring work.

The pull request included the template pattern changes, strategy pattern implementation, and the Random Walk feature.

I also added review comments to simulate a real code review process before merging the changes.